

NBA Player Points Prediction

By: Usman Tahir

Predicting per-game scoring using 2024-25 season - current data

Linear Regression · Random Forest · XGBoost

Motivation & Problem Definition

The Goal

Predict how many points a specific NBA player will score in their next game, before it happens.

Input: Historical player stats, efficiency metrics, rest days, opponent context, player identity

Output: Predicted point total for one player's upcoming game

Key rule: Only use information available before tip-off (no same-game stats)

Why It Matters

Fantasy Sports

Per-game scoring predictions directly inform daily lineup decisions worth real money

Sports Betting

Player prop markets (over/under points) rely entirely on scoring projections

Team Analytics

Understanding scoring variance helps coaches manage usage and rest

Dataset & KDD Pipeline

2024-25

NBA Season

312

Rotation Players (≥ 20 min avg)

28,824

Train rows $\rightarrow$ Oct 2024 – Dec 2025

80 / 20

Train / Test Split

1. Data Collection

Accurate NBA player game logs from 1947 to current day. Dataset was found on Kaggle and verified manually

2. Preprocessing

Filter season dates, drop DNP rows, remove bench players (< 20 min avg), drop redundant columns (made shots)

3. EDA

Univariate/bivariate distributions, per-player rolling trends, correlation analysis, archetype visualization

4. Feature Engineering

78 rolling features + efficiency metrics + contextual + KMeans archetypes + opponent encoding + player identity $\rightarrow$ 175 columns total

5. Feature Selection

Correlation filter ($r > 0.95$) dropped 13. VIF filter (> 10) dropped 87. Protected: roll3_pts_mean, player features.

6. Modeling & Evaluation

Baseline, Linear Regression, Random Forest, XGBoost (GridSearchCV). Time-based 80/20 split. Per-player prediction analysis.

Feature Engineering

Rolling Features — core predictive signal

13 base stats rolled over 3, 5, and 10 game windows:

points · minutes · rebounds · assists · FGA · FTA · 3PA · FG% · 3P% · FT% · TS% · eFG% · pts/min

roll_*_mean

Average over window — captures recent scoring level

roll5_*_std

Standard deviation — captures game-to-game consistency

roll5_*_max

Peak performance — captures ceiling scoring ability

roll5_*_slope

Linear trend — captures hot/cold momentum direction

**All rolling features use .shift(1) which is computed from previous games only.
No same-game stats.**

Efficiency Metrics

$TS\% = pts / (2 \times (FGA + 0.44 \times FTA))$

$eFG\% = FG\% + 0.5 \times (3PA / FGA) \times 3P\%$

Usage proxy = $FGA + 0.44 \times FTA + TOV$

Contextual Features

days_rest · is_b2b · games_played · home
usage_x_rest (fatigue × volume interaction)

Opponent Context

29 opponent OHE columns
opp_avg_pts_allowed (train period only)

Player Identity (new)

player_encoded — label-encoded personId
player_pts_std — historical scoring volatility
player_pts_max — personal season high so far

Feature Selection: Correlation Filter + VIF

Reduction Pipeline

175 candidate features

After all engineering

Correlation filter ($r > 0.95$)

Dropped 13 redundant features

VIF filter (> 10)

Dropped 87 multicollinear features

Protected list

roll3_pts_mean, player_encoded, player_pts_std,
player_pts_max — kept despite high VIF

~52 final features

Clean, non-redundant feature set for models

Why VIF?

Variance Inflation Factor (VIF)

Measures how well one feature can be predicted using all the others. High VIF = the feature is redundant and is already captured by something else.

$VIF = 1/(1 - R^2)$ where R^2 is from regressing that feature on all others

Why it matters here:

roll3_pts_mean, roll5_pts_mean, and roll10_pts_mean are all rolling averages of points (highly correlated). VIF flagged all three (VIF > 1000). We kept only roll3 since it's the most recent and most predictive.

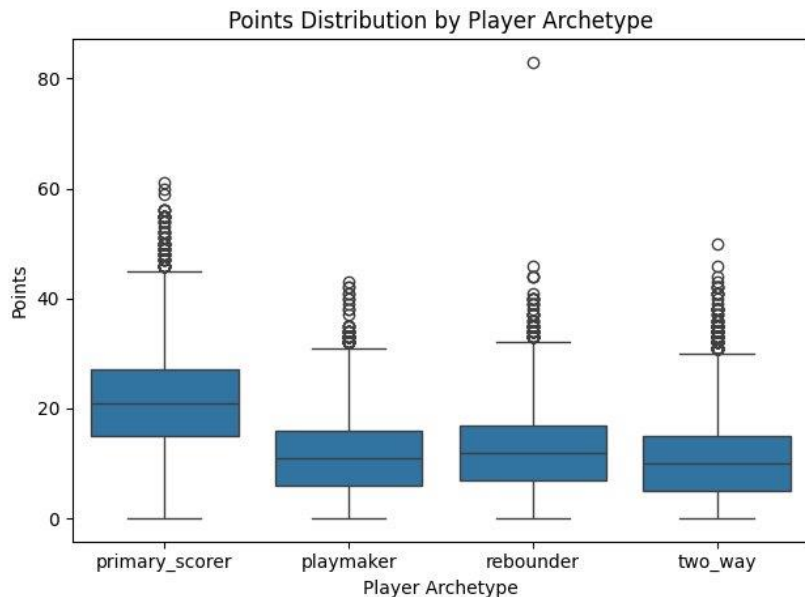
Also protected: player_encoded, player_pts_std, player_pts_max all carried high VIF due to per-player consistency, but are essential for individual player prediction.

VIF = 1 → Unique, keep

VIF > 10 → Redundant, candidate for removal

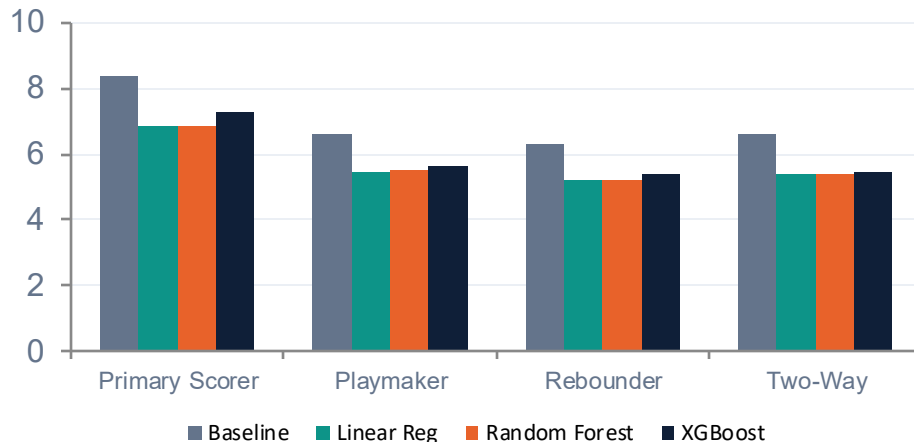
New Analysis: Per-Archetype Error Breakdown

Actual Points Distribution by Archetype



Primary scorers have the widest spread and tallest whiskers — their scoring is inherently unpredictable. High model error for this archetype reflects real variance in the data, not model failure.

Test RMSE by Player Archetype



Key Findings

Primary scorers: hardest to predict RMSE 6.8–8.4 vs 5.2–6.6 for others — but the boxplot shows why: their actual scoring swings wildly game-to-game.

Rebounders: best RMSE (5.21) Consistent role = consistent scoring. The boxplot confirms a tight distribution.

All models beat baseline on every archetype Feature engineering adds real value across all player types.

Algorithms & Methods

Baseline

BL

Predicts each player's 5-game rolling average. No learning — purely a minimum bar to beat. Any model that cannot beat this adds no value over naive history.

No parameters · No scaling · Pure lookahead

Linear Regression

LR

Learns a weight for each feature assuming additive, straight-line relationships. Sensitive to scale and feature distribution. Required StandardScaler + log1p transform of skewed features.

StandardScaler · log1p on 13 skewed features · fit on training period only

Random Forest

RF

Ensemble of 200 decision trees, each trained on random feature/sample subsets. Averages all predictions. Handles non-linear interactions — no scaling needed.

n_estimators=200 · max_depth=10 · min_samples_leaf=5

XGBoost (new —
GridSearchCV
tuned)

XGB

Sequential boosting — each tree corrects errors of previous trees. Tuned with 5-fold GridSearchCV. Best params: learning_rate=0.1, max_depth=4, min_child_weight=3, subsample=0.9.

n_estimators=500 · colsample_bytree=0.8 · Best CV RMSE: 5.684

Model Results

Model	Train RMSE	Test RMSE	Test MAE	Test R ²
Baseline	7.061	7.090	5.522	0.309
Linear Reg	5.743	5.813	4.520	0.536
Random Forest	5.065	5.805	4.514	0.537
XGBoost	4.946	6.017	4.663	0.503

7.09 → 5.81 RMSE

All 3 models beat baseline

LR & RF tied (~5.81)

Best overall

$R^2 \approx 0.54$

Variance explained

Train 4.95 → Test 6.02

XGBoost overfits

Player-Specific Prediction: Ryan Rollins

Ryan Rollins

Playmaker

30

Test games

#1 RF

RF R² ranking

0.436

Best RF R²

Actual vs Predicted — Test Period (Dec 2025 – Mar 2026)

Date	Actual	Baseline	Linear Reg	Rand Forest	XGBoost
2025-12-29	13	18.6	18.0	19.4	19.0
2025-12-31	16	16.2	19.5	21.1	20.1
2026-01-02	29	15.6	21.1	22.2	21.7
2026-01-04	12	20.6	20.4	20.0	19.2
2026-01-07	16	19.6	9.4	11.2	10.5
2026-01-09	10	19.6	8.6	9.4	9.1
2026-01-11	2	15.8	8.7	9.5	8.8
2026-01-23	21	8.6	20.2	19.7	21.7
2026-01-27	24	12.4	22.8	24.0	26.4
2026-02-01	25	17.2	22.2	22.5	24.2
2026-02-04	27	21.6	22.8	23.6	24.8
2026-03-01	11	20.0	12.6	11.1	12.3

Linear Reg RMSE 5.014 R² 0.420

Rand Forest RMSE 4.947 R² 0.436

XGBoost RMSE 5.025 R² 0.418

Per-Player R² Rankings Across All Models

Player	Archetype	Games	LR R ²	RF R ²	XGB R ²
Jusuf Nurkic	Rebounder	12	0.495	0.403	0.475
Devin Booker	Primary Scorer	24	0.471	0.317	-0.058
Ryan Rollins	Playmaker	30	0.420	0.436	0.418
Nikola Jovic	Two-Way	22	0.419	0.453	0.477
Coby White	Primary Scorer	19	0.419	0.333	0.320
Sam Hauser	Two-Way	31	0.401	0.421	0.326
Scottie Barnes	Primary Scorer	32	0.397	0.429	0.077
Tari Eason	Rebounder	27	0.381	0.326	0.318
LeBron James	Primary Scorer	33	0.345	0.328	0.434
Precious Achiuwa	Rebounder	33	0.345	0.401	0.400

Key insight: Ryan Rollins (Playmaker) is most consistent across all three models (0.420 / 0.436 / 0.418) with 30 test games. Two-way and rebounder archetypes dominate the top rankings. Best LR R² is 0.495 (Nurkic), though only 12 test games.

Conclusion & Future Work

Key Takeaways

- All three models beat the naive baseline (7.09 → 5.81 RMSE). Feature engineering adds real predictive value.
- Linear Regression and Random Forest are essentially tied for best overall performance ($R^2 \approx 0.54$), showing rolling features are primarily linear in nature.
- Player archetype drives how well the model works — rebounders and two-way players are most predictable, primary scorers least.
- Individual player R^2 (up to 0.495 for Nurkic) exceeds aggregate metrics, confirming the value of player-specific analysis.

Future Work

- Multiple seasons of data — one season limits per-player training rows. More data would directly improve individual player R^2 .
- Archetype-specific models — train a separate model per archetype to handle primary scorer volatility better.
- Opponent defensive ratings from external APIs — current OHE encoding only captures team identity, not defensive quality.
- Injury and load management flags — returning players and strategically rested stars introduce systematic prediction errors.